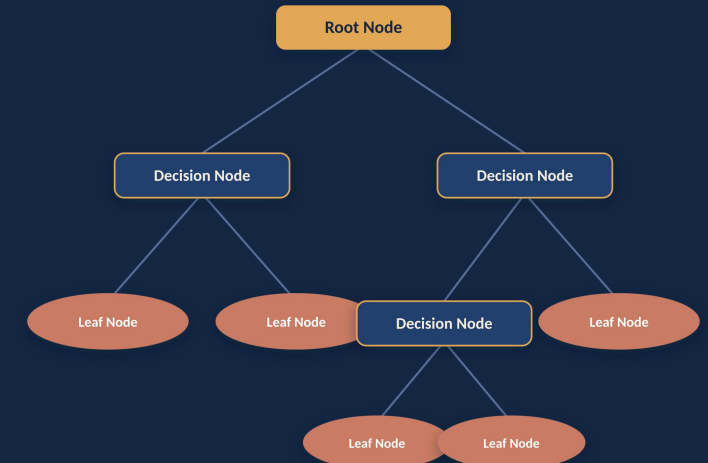


MACHINE LEARNING · SUPERVISED LEARNING

Decision Trees

From root to leaf: purity, entropy, and the geometry of every split



What Is a Decision Tree?

A supervised learning algorithm used for classification and regression. It behaves like a binary flowchart, making predictions by asking a sequence of simple yes/no questions about the data's features.

Root Node

The topmost box containing 100% of training data before any splits.

Decision Nodes

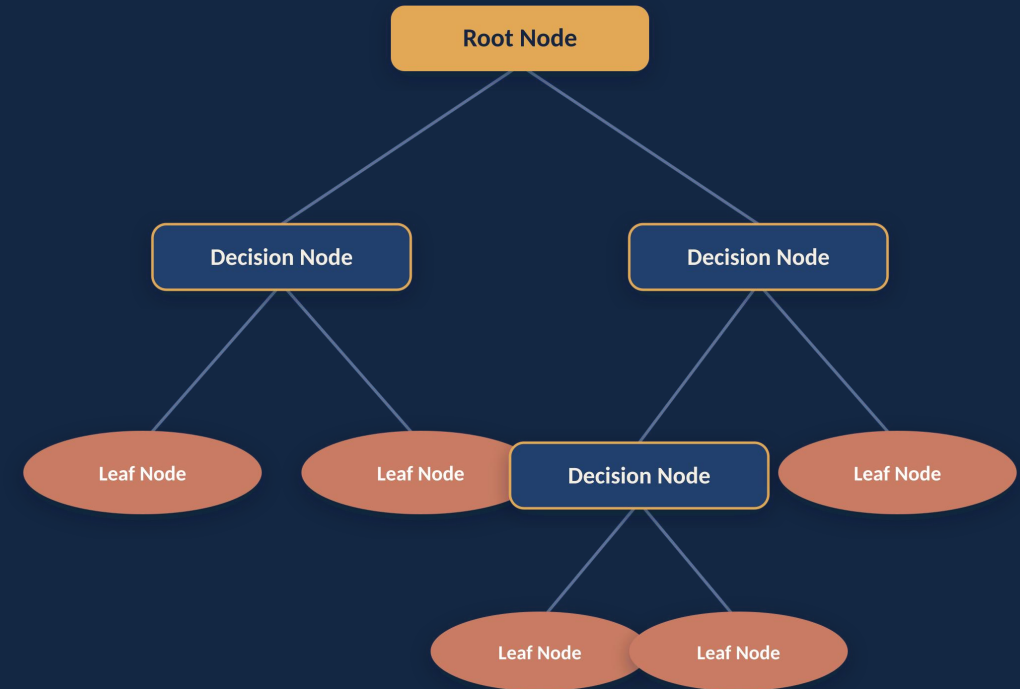
Intermediate boxes testing a condition on a single feature.

Branches

The Yes/No paths taken based on the answer to a question.

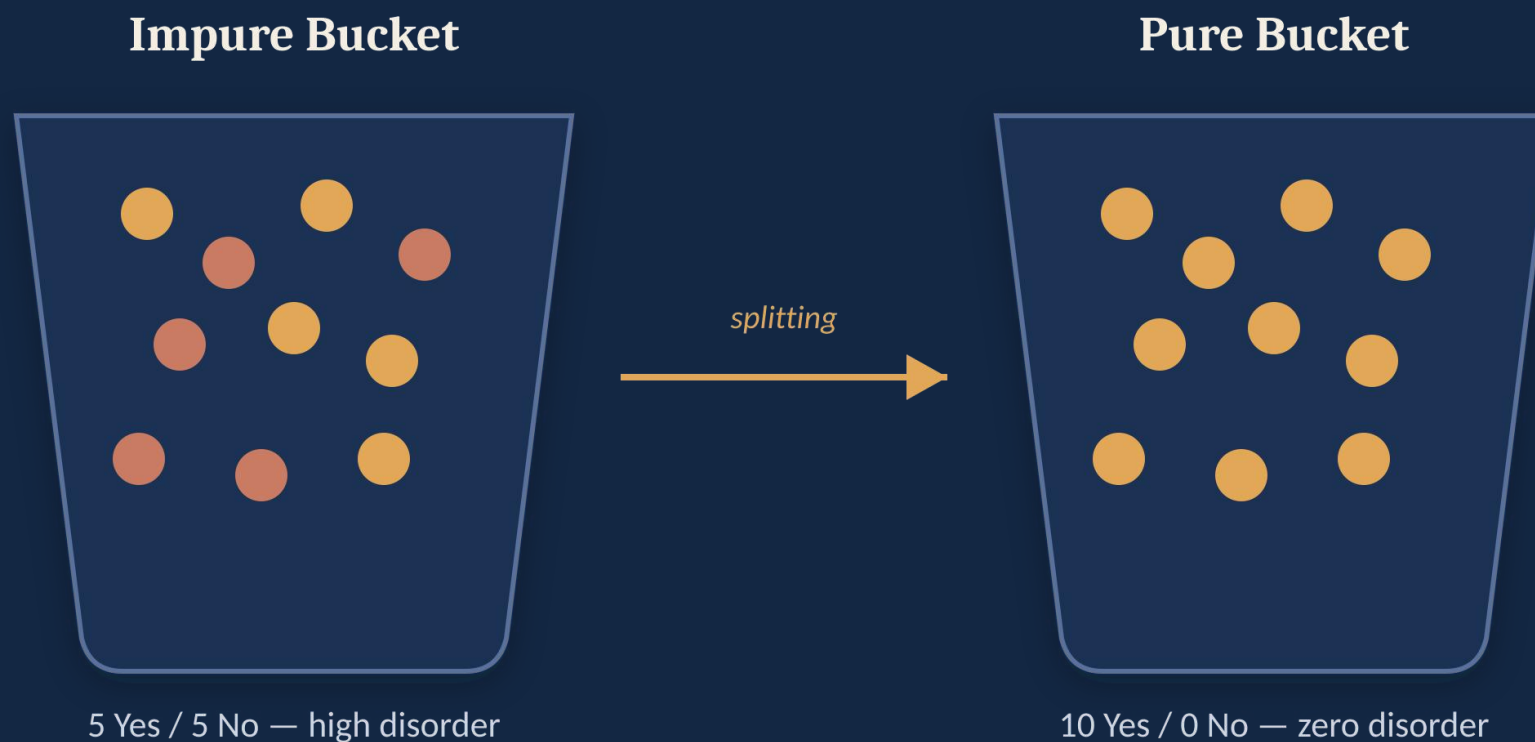
Leaf Nodes

Terminal boxes holding the final prediction — a class or a number.



Purity vs. Impurity

Every split exists to transform a messy, mixed pile of data into pure subsets. An impure (50/50) bucket means maximum uncertainty — a prediction is a coin toss. A pure bucket means every sample agrees, so predictions can be made with confidence.



Entropy: Information Disorder

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i)$$

Binary (2 classes)

Min = 0.0 → pure node. Max = 1.0 → 50/50 split.

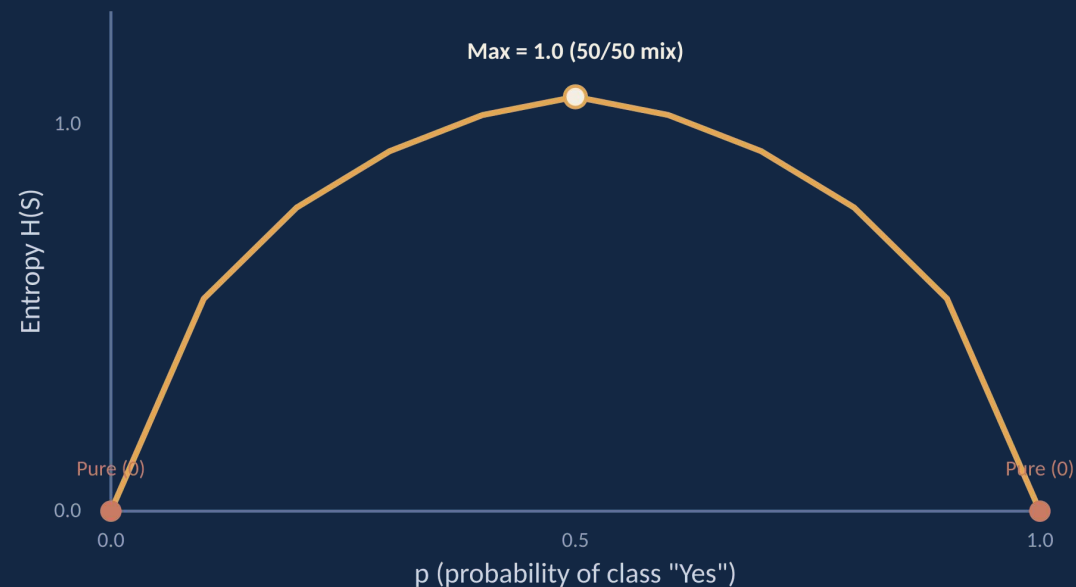
Multi-class (K classes)

Min = 0.0 → pure. Max = $\log_2(K)$, all classes equal.

Example (K = 3)

Max entropy = $\log_2(3) \approx 1.58$

Narrow, peaked distributions → low entropy. Wide, flat distributions → high entropy.



Gini Impurity

Gini impurity measures the probability of misclassifying a randomly chosen element if it were labeled according to the class distribution of the node. It's faster to compute than entropy and gives very similar splits in practice.

$$\text{Gini}(S) = 1 - \sum p_i^2$$

Entropy

Min = 0.0

Max = 1.0 (2 classes)

Logarithmic — slightly more expensive to compute

Gini Impurity

Min = 0.0

Max = 0.5 (2 classes)

Purely quadratic — the scikit-learn default

Both metrics agree on which splits reduce disorder — they rank candidate questions almost identically in practice.

How the Tree Selects Every Split

1

Categorical features

Test discrete equality directly (e.g., Weather == Sunny?).

2

Numerical features

Sort unique values, take midpoints between neighbors, test each as a threshold.

3

Evaluate candidates

Compute the entropy of the resulting child buckets for every feature and split point.

4

Calculate Information Gain

$IG = \text{Entropy}(\text{Parent}) - \text{Weighted Average Entropy}(\text{Children})$.

5

Pick the winner

The question with the highest Information Gain becomes the decision node.

6

Recurse

Repeat the entire process independently on each child subset.

Information Gain

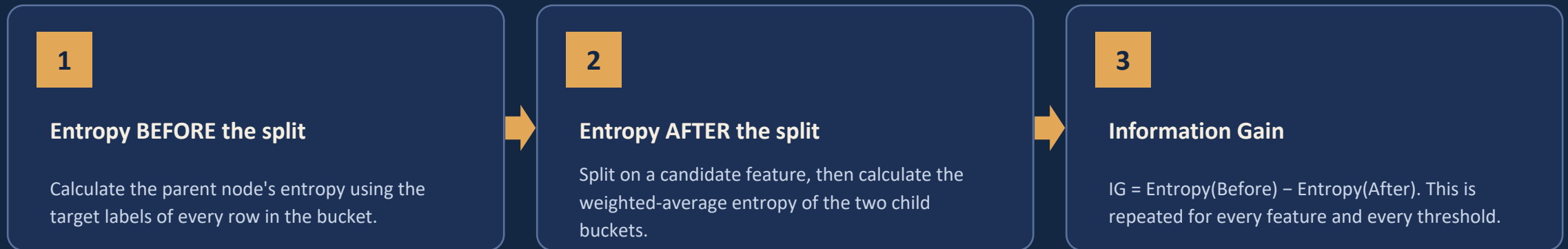
$$IG = H(S) - [(|S_l|/|S|) \cdot H(S_l) + (|S_r|/|S|) \cdot H(S_r)]$$

Execution order

The algorithm tests every candidate question at the root, locks in the single best one, then physically splits the data. It moves to the left branch, repeats the full search on that subset alone, attaches the winning node, then does the same on the right branch — continuing node-by-node until a stopping condition is reached.

How the Model Actually Picks a Feature

For every candidate question, the model compares entropy before and after the split — then keeps whichever question shrinks disorder the most.



The selection rule

Big drop in entropy → high Information Gain → this question wins the node.

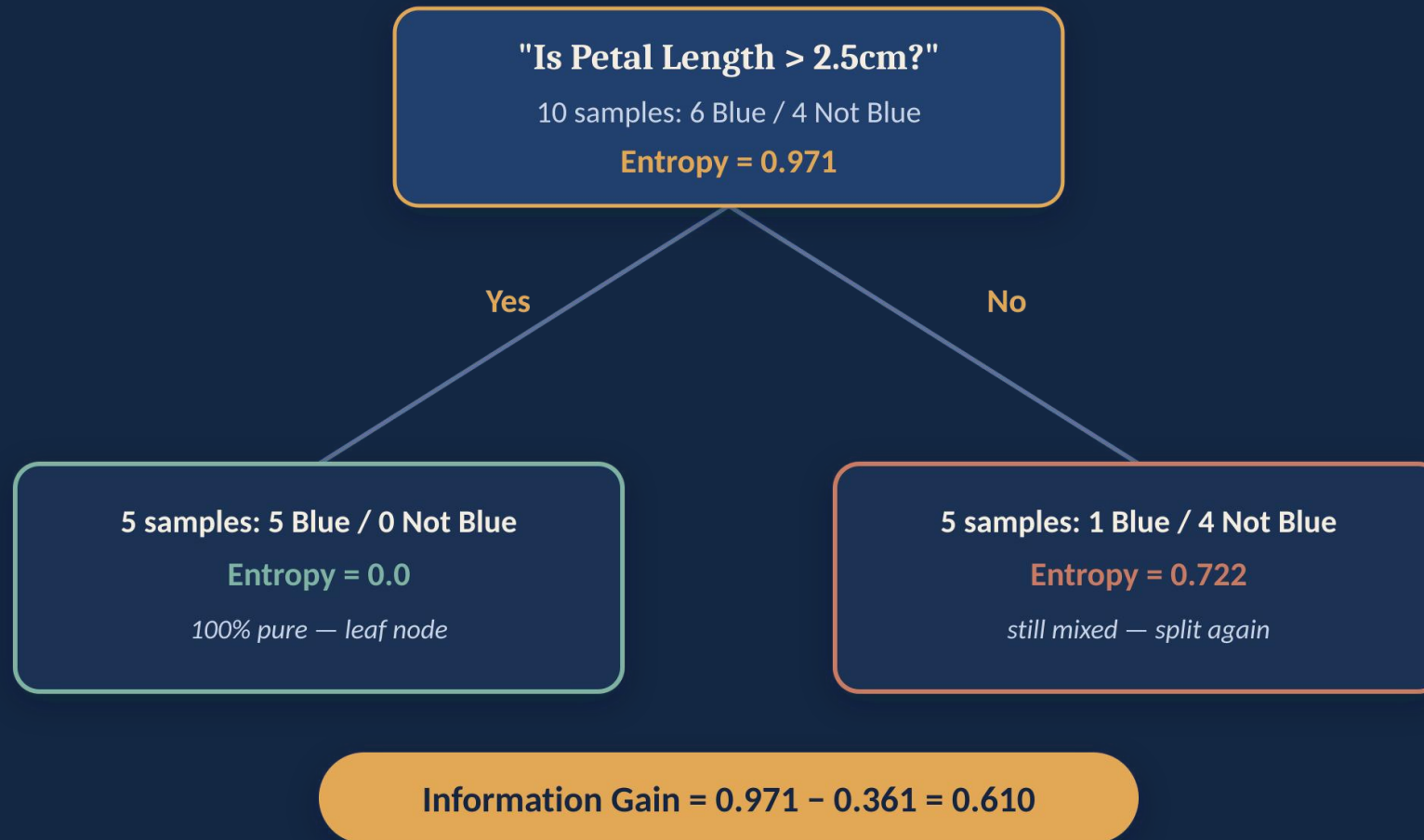
Small drop in entropy (children still messy) → low Information Gain → this question is discarded and the next candidate is tried.

Across every candidate feature and threshold, the single split with the highest Information Gain becomes the decision node.

WORKED EXAMPLE

Information Gain, Step by Step

10 flowers → 6 Blue, 4 Not Blue. Candidate question: "Is Petal Length > 2.5 cm?"



The Model Never “Asks” About the Label

The tree only ever questions features. The target column already sits in the training table — the model just counts it.

Flower	Petal Length (Feature)	Is Blue? (Target)
Flower 1	1.0 cm	Yes
Flower 2	1.2 cm	Yes
Flower 3	1.5 cm	Yes
Flower 4	1.8 cm	Yes
Flower 5	2.0 cm	Yes
Flower 6	2.8 cm	Yes
Flower 7	3.0 cm	No
Flower 8	3.5 cm	No
Flower 9	4.0 cm	No
Flower 10	4.2 cm	No

Left Bucket — “Petal Length ≤ 2.5cm?” = Yes

Flowers 1–5 → scan target column: 5 Yes / 0 No

100% pure → Leaf Node predicts “Blue”

Right Bucket — “Petal Length ≤ 2.5cm?” = No

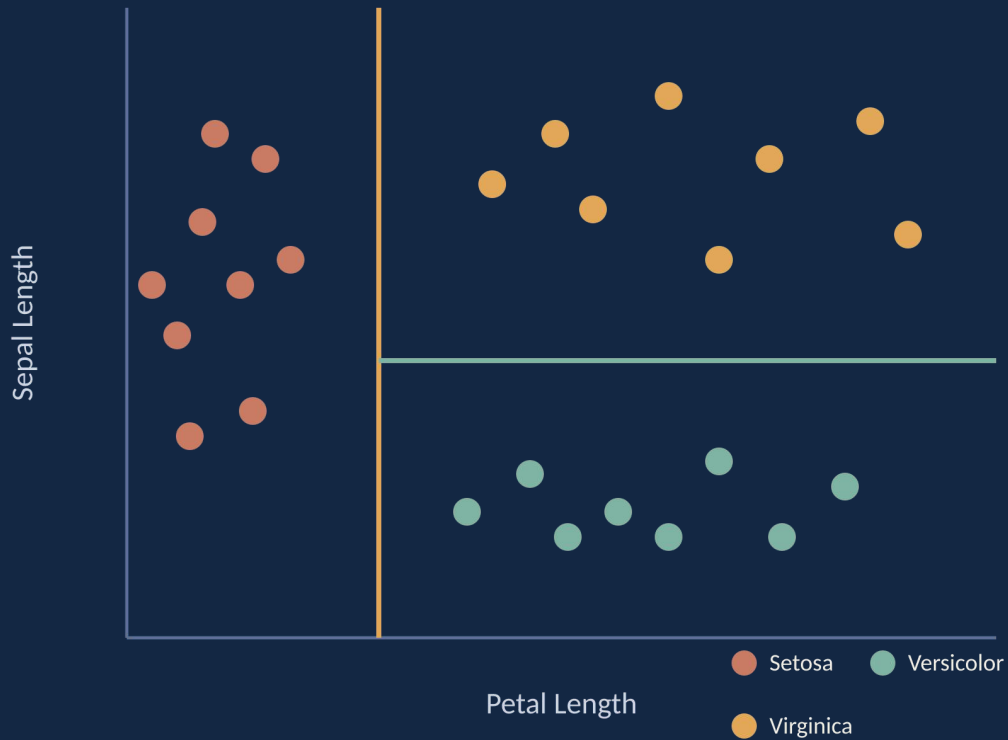
Flowers 6–10 → scan target column: 1 Yes / 4 No

Still mixed → Entropy = 0.722, split again

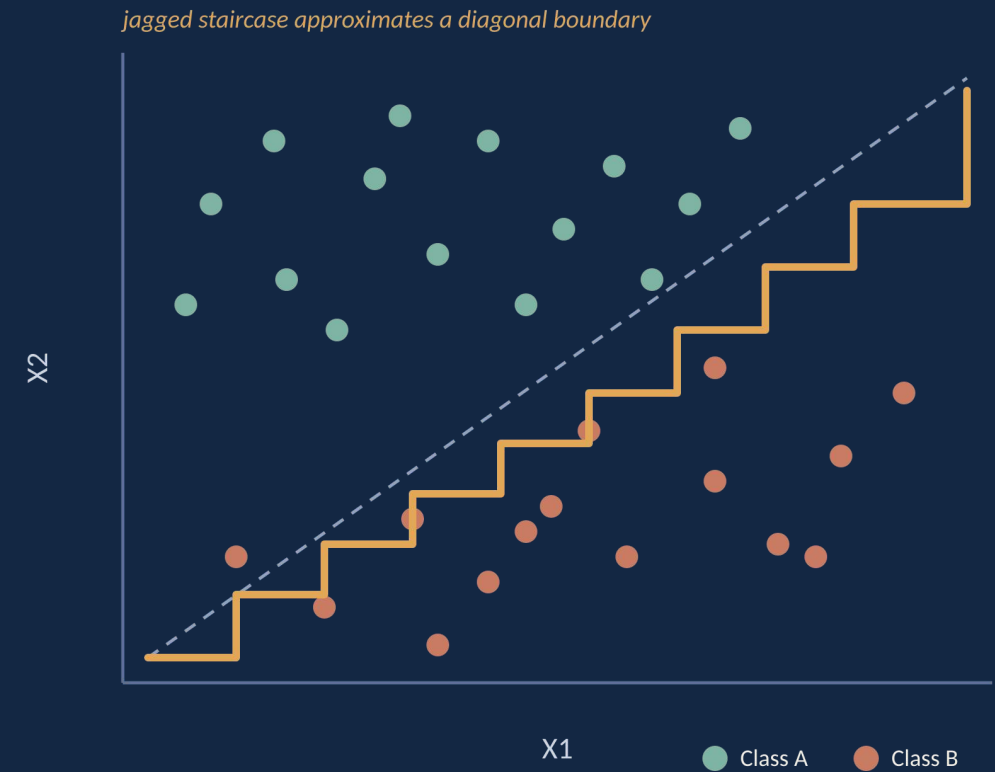
Nodes test FEATURES only. Target labels are never asked — they're counted from the table, instantly, in memory.

Splits Are Always Axis-Aligned

Every split evaluates one feature against a constant ($X_i > c$). In 2D this draws horizontal or vertical lines; in N-D it forms orthogonal hyperplanes carving the space into rectangular boxes.

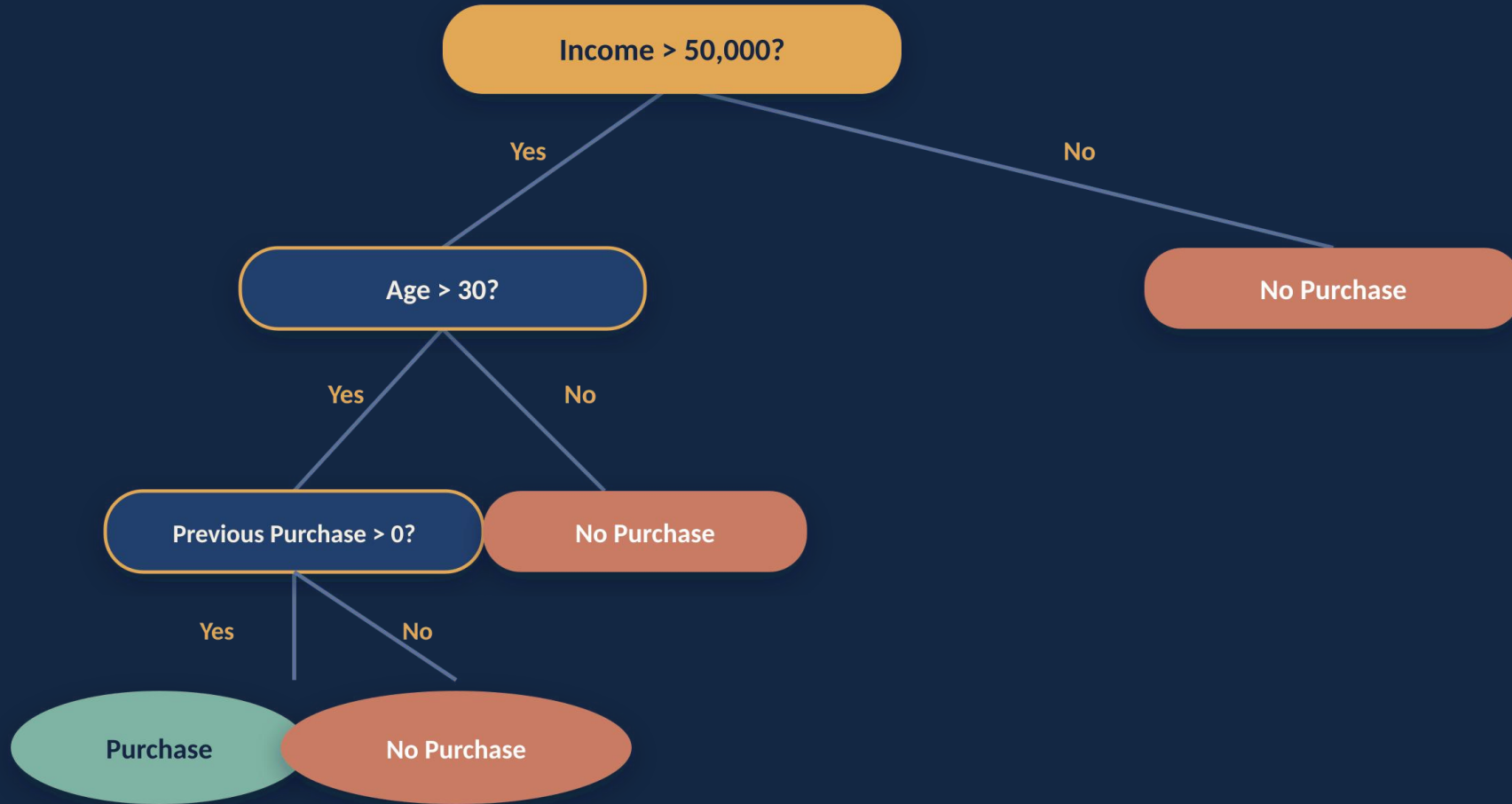


Iris dataset: two axis-aligned cuts separate all three species



Diagonal boundary → the tree must approximate it with a staircase of cuts

Predicting Whether a Customer Will Buy



Real-World Applications



Finance

Credit scoring, loan approval, and fraud flagging based on applicant features.



Healthcare

Diagnosis support and patient risk stratification from symptoms and vitals.



Marketing

Predicting customer churn, purchase likelihood, and campaign targeting.



Manufacturing

Quality control and defect detection on production line sensor readings.



Human Resources

Resume screening and attrition-risk prediction from employee data.



Ensemble Models

The core building block inside Random Forests and Gradient Boosting (XGBoost).

Six Key Limitations



Staircase Boundaries

Diagonal true boundaries force deep, complex axis-parallel approximations.



Class Imbalance

With 95/5 splits, predicting the majority class already looks “pure” — the tree ignores rare cases.



High-Dimensional Data

Thousands of sparse columns dilute Information Gain across weak, noisy features.



Overfitting

Unconstrained trees memorize training noise until every leaf is 100% pure.



Instability

A few changed data points can flip the winning root feature entirely.



No Extrapolation

Leaves output fixed values — trees can't predict beyond the training data's range.

Two Families of Pruning Penalties

Decision tree penalties split into two families based on WHEN they intervene in the tree's growth.



Pre-Pruning

Early Stopping

Rules applied DURING construction that prevent a split from happening in the first place — like a security guard checking every proposed split before it's built.

- `max_depth` — caps tree height
- `min_samples_split` — minimum parent size
- `min_samples_leaf` — minimum child size
- `min_impurity_decrease` — minimum gain required



Post-Pruning

Cost-Complexity

Rules applied AFTER the tree has fully grown, mathematically penalizing complexity to chop off branches that don't pull their weight.

`ccp_alpha`

The only pure mathematical penalty score in decision trees — it works like Lasso (L1) or Ridge (L2) regularization in linear regression.

Four Stopping Rules During Growth



max_depth

Depth Penalty

Caps how many levels deep the tree can grow. Once the limit hits, no more questions — even on a messy leaf.

max_depth = 3 stops overly specific rules like “income between \$45,100–\$45,200”



min_samples_split

Parent Node Penalty

Minimum samples a node must hold to even attempt a split.

18 samples < threshold of 20 → forced into a leaf, even though a clean 10/8 split exists



min_samples_leaf

Child Node Penalty

Minimum samples required in EACH resulting child bucket after a split.

A split producing a 2-sample bucket is rejected if min = 10 — even with high Information Gain



min_impurity_decrease

Gain Threshold Penalty

Requires a split to improve purity by at least a set minimum amount.

IG = 0.02 is below the required 0.05 threshold → the split is rejected

Cost-Complexity Pruning (ccp_alpha)

The only pure mathematical penalty score in decision trees — it works like Lasso or Ridge regularization does in linear regression.

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

$R(T)$

Total Impurity

The total misclassification rate or summed entropy across all leaf nodes in tree T .

$|T|$

Tree Size

The total number of leaf nodes — a direct, countable measure of complexity.

α

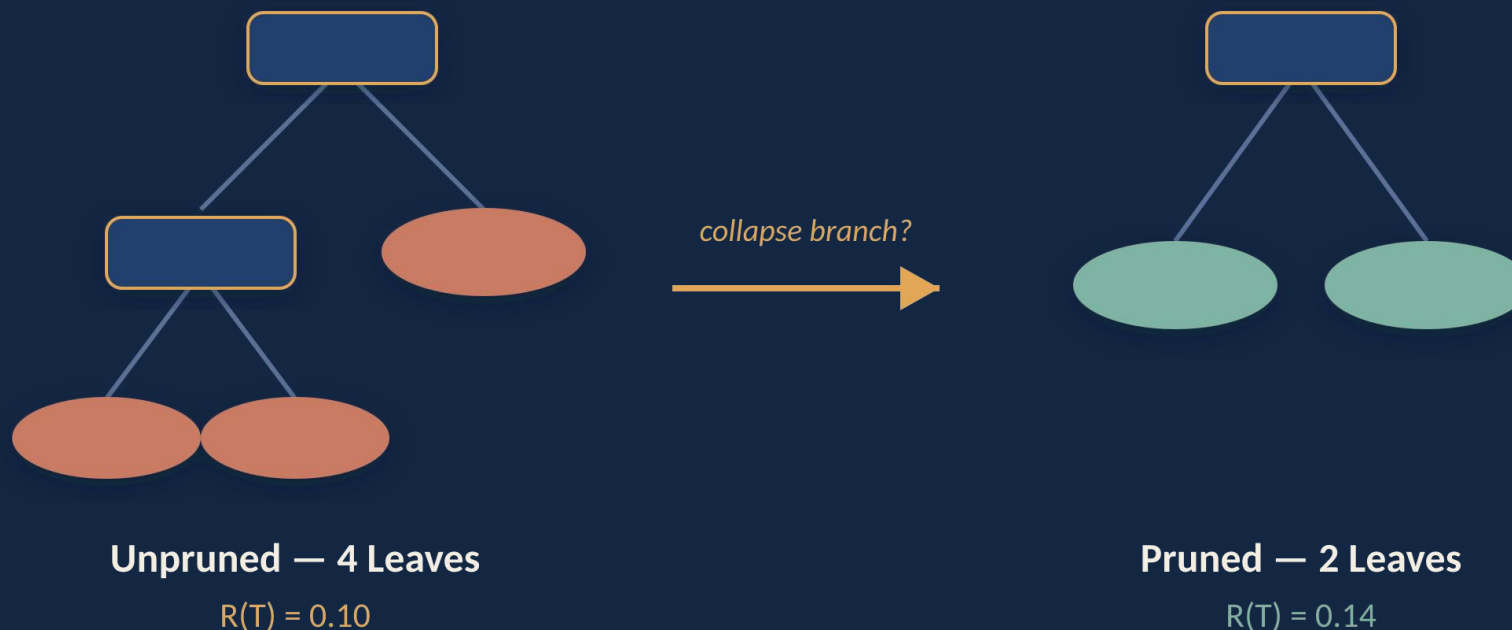
Penalty Multiplier

Set by you (ccp_alpha). Controls how heavily extra leaves get penalized.

WORKED EXAMPLE

When Does Pruning Actually Happen?

A branch with 4 leaves ($R = 0.10$) could collapse into 2 leaves ($R = 0.14$). Does it get pruned?



Scenario A — $\alpha = 0.01$ (low penalty)

4 leaves: $0.10 + 0.01 \times 4 = 0.14$ | 2 leaves: $0.14 + 0.01 \times 2 = 0.16$

$0.14 < 0.16 \rightarrow$ No pruning occurs

Scenario B — $\alpha = 0.05$ (high penalty)

4 leaves: $0.10 + 0.05 \times 4 = 0.30$ | 2 leaves: $0.14 + 0.05 \times 2 = 0.24$

$0.24 < 0.30 \rightarrow$ Branch is pruned!

Penalty Comparison Matrix

Penalty	Stage	What It Measures	Best Used For
<code>max_depth</code>	Pre-pruning	Maximum vertical height of the tree	Quick baseline control against extreme overfitting
<code>min_samples_split</code>	Pre-pruning	Samples required in a parent node to allow splitting	Stopping early on small subsets of data
<code>min_samples_leaf</code>	Pre-pruning	Minimum samples allowed in any new child leaf	Preventing noisy outliers from getting their own leaf
<code>min_impurity_decrease</code>	Pre-pruning	Minimum Information Gain required to justify a split	Filtering out weak features with almost no purity gain
<code>ccp_alpha</code>	Post-pruning	Mathematical penalty per leaf: $R(T) + \alpha T $	Fine-tuning a fully grown tree on noisy or small datasets

Matching Penalty to Problem Type

The problem type, dataset size, and target variable directly dictate which strategy to start with.



Small or Noisy Dataset

`ccp_alpha` (Post-Pruning)

Pre-pruning may cut a branch before it finds a real pattern. Let the tree grow fully, then prune weak leaves.



High-Dimensional / Sparse Data

`min_samples_leaf`

Forces every leaf to represent a meaningful cluster — not a lucky 1–2 point split among thousands of columns.



Imbalanced Classification

`class_weight='balanced' + small min_samples_leaf`

A strict `min_impurity_decrease` would kill the tiny-IG splits that isolate the rare class you actually care about.



Regression (Continuous Targets)

`max_depth + min_samples_leaf`

Stops thousands of single-sample leaves from creating jagged, high-variance step functions.

Why Tree Penalties Aren't Like Ridge / Lasso

Linear models penalize weights (β) directly. Trees have no weights — penalties instead control how deep, complex, or fine-grained the rules are allowed to get. A useless feature is simply never picked for a split; feature selection happens automatically.

Data Situation	Linear Model Equivalent	Best Tree Penalty
Too many columns / sparse features	Lasso (L1)	<code>min_samples_leaf</code>
Noisy / small dataset	Ridge (L2)	<code>ccp_alpha</code> (Post-Pruning)
Tree too deep / memorizing data	Weight decay	<code>max_depth</code>
Splits giving tiny, useless gains	Thresholding	<code>min_impurity_decrease</code>

Trees do feature selection naturally — penalties just stop the tree from over-analyzing subsets of data.

Advantages vs. Disadvantages

Advantages

- Intuitive and easy to understand — mirrors human decision-making
- Minimal data preparation required — no scaling or normalization needed
- Inference cost is logarithmic in the number of training data points
- Handles both numerical and categorical features natively

Disadvantages

- Prone to overfitting without depth limits or pruning
- Prone to errors on imbalanced datasets
- Unstable — small data changes can restructure the whole tree
- Cannot extrapolate beyond the training data's range